Longest Substring Without Repeating Characters

Two approaches, a frame-by-frame walkthrough, and the core logic to remember

Brute Force · O(n³) **Sliding Window · O(n)**

Reference notes · LongestSubStringWithoutRepeat.java

Problem Statement

Given a string `s`, return the length of the longest **substring** that contains no duplicate characters.

s = "pwwkew" → **longest window with no repeated char** → **Output: 3 ("wke")**

Approach Comparison

Approach	Time	Space	Core Idea
Brute Force	O(n³)	O(min(n, m))	Check every substring, re-scan it with a fresh set
Sliding Window	O(n)	O(min(n, m))	Duplicate-free window; shrink from the left on each duplicate

Big picture: the window `[low, high)` is *always* duplicate-free, so a `Set` (presence only, no counts) is enough. Each character enters the set once and leaves at most once — that is why it is O(n) even with the nested `while`.

How to read the walkthrough: a PDF can't play animation, so the sliding window is shown as a **storyboard** — every frame is one tick of the loop. Read left → right, top → bottom, like a flipbook.

Frame legend

not visited · inside window · just added · being examined · duplicate · dropped (outside) · best window

L = low pointer (window start) **H** = high pointer (char being processed)

1 · Brute Force

O(n³) time **O(min(n, m)) space**

```java
// every start i, every end j, re-scan s[i..j] for duplicates
private static int usingBruteForce(String s) {
    int maxLength = 0;

    for (int i = 0; i < s.length(); i++) {
        for (int j = i; j < s.length(); j++) {

            Set<Character> set = new HashSet<>();
            boolean isUnique = true;

            for (int k = i; k <= j; k++) {
                char ch = s.charAt(k);

                if (set.contains(ch)) {
                    isUnique = false;
                    break;
                }
                set.add(ch);
            }

            if (isUnique) {
                maxLength = Math.max(maxLength, j - i + 1);
            }
        }
    }
    return maxLength;
}
```

Recall trick: "Pick every start, pick every end, re-scan the middle for duplicates." Three nested loops → O(n³). Correct, but it re-checks the same characters over and over.

2 · Sliding Window — the code

O(n) time **O(min(n, m)) space**

```java
// duplicate arrives? shrink from the left until it's gone, then add it
private static Integer usingSlidingWindow(String s) {
    HashSet<Character> set = new HashSet<>();
    int high = 0;
    int low = 0;
    int maxLength = 0;

    while (high < s.length()) {

        while (set.contains(s.charAt(high))) {
            set.remove(s.charAt(low));
            low++;
        }

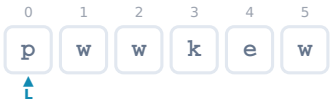
        set.add(s.charAt(high));
        high++;
        int currentLength = high - low;
        maxLength = Math.max(maxLength, currentLength);
    }
    return maxLength;
}
```

Recall trick: "Duplicate arrives? Shrink from the left until it's gone, then add it." It is a `while`, not an `if` — you may need to evict several characters before the duplicate itself leaves.

Sliding Window — frame-by-frame walkthrough

s = "pwwkew" | read like a flipbook: left → right, top → bottom

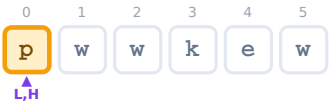
1 Start — START



set = ø | len **0** | max **0**

Empty window, empty set. **L** (low) marks the window start.

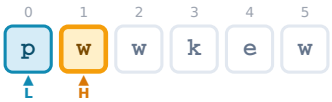
2 'p' arrives — ADD



set = p | len **1** | max **1** ★ **new max**

'p' is not in the set → add it. Window grows to length 1.

3 'w' arrives — ADD



set = p w | len **2** | max **2** ★ **new max**

'w' is not in the set → add it. Window *pw*, length 2.

4 'w' arrives again — DUPLICATE



set = p w | len **2** | max **2**

'w' is **already in the set** → duplicate! Shrink from the left until it's gone.

5 Shrink: drop 'p' — SHRINK



set = w | len **1** | max **2**

Remove s[low] = 'p', low → 1. 'w' is *still* in the set, so keep shrinking.

6 Shrink: drop 'w' — SHRINK



set = ø | len **0** | max **2**

Remove s[low] = 'w', low → 2. No 'w' left in the set → stop shrinking.

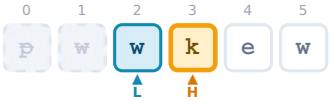
7 Add 'w' — ADD



set = w | len **1** | max **2**

Now it is safe to add 'w'. Window = [2,2], length 1. Max stays 2.

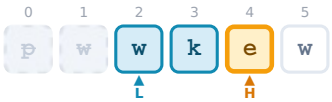
8 'k' arrives — ADD



set = w k | len **2** | max **2**

'k' is not in the set → add it. Window *wk*, length 2.

9 'e' arrives — ADD



set = w k e | len **3** | max **3** ★ **new max**

'e' is not in the set → add it. Window *wke*, length 3 — new record!

10 'w' arrives again — SHRINK



set = k e | len **2** | max **3**

'w' is a duplicate → remove s[low] = 'w', low → 3. Conflict gone.

11 Add 'w' — end — DONE



set = k e w | len **3** | max **3**

Add 'w'. Window *kew*, length 3. End of string → answer = **3**.

✓ Result — RESULT



longest window length = **3**

Longest duplicate-free window = **"wke"** (length **3**); "kew" ties it.

TL;DR — For Future Me

- **Best solution:** variable-size sliding window + `Set`, O(n) — duplicate arrives? shrink from the left until it's gone, then add and record the length.
- **Brute force** is O(n³) — every substring, each re-scanned from scratch.
- **Must remember:** the shrink step is a `while` loop, not an `if`.
- **Pattern:** a `Set` suffices when the window must have *no repeats*; a `Map` of counts is needed when repeats are allowed (K-distinct, Fruit Into Baskets).